

Disjoint Set Union on Grids

Grid Overview

Suppose grid is

1	1	0
0	1	0
1	0	1

Rows = 3

Cols = 3

Node Conversion Logic

Every cell becomes one node.

```
node = row * m + col;
```

Current node

```
curr = r * m + c;
```

Neighbour node

```
next = nr * m + nc;
```

Simply

```
union(curr, next);
```

Mapping Visualization

Grid

1	1
1	0

Nodes

0	1
2	3

Implementation

```
class dsu{
public:
    vector<int> parent;
    vector<int> size;

    dsu(int n){
        parent.resize(n);
        size.resize(n, 1);

        for(int i=0; i<n; i++){
            parent[i] = i;
        }
    }

    int findparent(int node){
        if(parent[node] == node){
            return node;
        }
        parent[node] = findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u, int v){
        int pu = findparent(u);
        int pv = findparent(v);

        if(pu == pv) return;

        if(size[pu] < size[pv]) swap(pu, pv);

        parent[pv] = pu;
        size[pu] += size[pv];
    }
}
```

```

};

class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int n = grid.size();
        int m = grid[0].size();

        vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};
        dsu ds(n*m);

        for(int r=0; r<n; r++){
            for(int c=0; c<m; c++){
                if(grid[r][c] == '0') continue;
                int curr = r*m+c;

                for(auto &it : dir){
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if(nr>=0 && nc>=0 && nr<n && nc<m){
                        int next = nr*m+nc;
                        if(grid[nr][nc] == '1'){
                            ds.unionit(curr, next);
                        }
                    }
                }
            }
        }

        unordered_set<int> st;
        for(int r=0; r<n; r++){
            for(int c=0; c<m; c++){
                if(grid[r][c] == '0') continue;
                int curr = r*m+c;
                st.insert(ds.findparent(curr));
            }
        }

        return st.size();
    }
};

```